

SBA301 – SLOT 09

Chapter 08: Fetching and Caching Data

Promise • async/await • Fetch • Axios • HTTP Caching • React Data Fetching

CLO2 – Apply fetching/caching data in Client–Server Communication

ReactJS • JavaScript • Browser DevTools • Fetch • Axios • JSONPlaceholder

Phạm vi: Chapter 08, slide 1–23 | Không đi sâu React Router/Spring Boot backend

Mental model xuyên suốt: User action → async request → Promise pending → HTTP response → validate status → parse data → update Loading/Error/Data → cleanup/cancel stale request → optionally reuse cached representation.

Engineering rule: Không đánh giá data fetching bằng “console có dữ liệu”. Phải chứng minh request, status, response, loading/error state, cleanup và behavior khi lỗi/race xảy ra.

CÁCH SỬ DỤNG GIÁO TRÌNH

Tài liệu theo mạch Prepare → Trace → Promise → Fetch/Axios → React State → Break → Cache → Verify → Transfer. Mỗi phần đi từ mental model đến code, DevTools evidence và câu hỏi giải thích.

Pha học	Sinh viên làm gì?	Bằng chứng
Prepare	Ôn HTTP/JSON/REST từ Slot 8, chạy React project.	Pre-check + baseline
Trace	Theo dõi Call Stack → Web API → queue → Promise callback.	Async trace
Fetch	Gọi API và kiểm tra status/body.	Network evidence
State	Mô hình hóa loading/error/data.	UI state matrix
Break	Tạo 404, slow request, malformed data, race.	Break-It log
Cache	Quan sát Cache-Control/ETag/304 và client cache đơn giản.	DevTools cache trace
Verify	Dùng cleanup/AbortController, reload, repeat.	Verification checklist
Transfer	Áp dụng vào Orchid/FUNews/Assignment.	Design note

AI rule: AI may generate code; student must run it, explain Promise/request/state flow, verify Network tab và chỉ ra behavior khi request lỗi hoặc bị hủy.

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC BUỔI NÀY

0.1 Ôn lại kiến thức Slot 8

- Client–Server Communication và request/response.
- HTTP methods: GET, POST, PUT, DELETE.
- Status code 2xx/4xx/5xx.
- JSON và REST resource/endpoint.
- DevTools Network: method, URL, status, headers, response.

0.2 Kiến thức JavaScript/React cần nhớ

- Function, arrow function, callback.
- Array.map/filter; object destructuring.
- React function component, useState, useEffect.
- State update gây render lại; side effect không nên chạy trực tiếp trong render.

0.3 Công cụ và tương thích môi trường

Thành phần	Khuyến nghị	Ghi chú
Node.js	Theo syllabus: Node 21	Node 21 không phù hợp với Vite hiện hành.
Nếu giữ Node 21	Pin Vite 5 cho project thực hành	Tránh `@latest` trên máy phòng lab cũ.
Nếu dùng Vite hiện hành	Node 20.19+ hoặc 22.12+	Ưu tiên Node 22 LTS trong máy cá nhân mới.
Browser	Chrome/Edge	Network + Console + Disable cache
API	JSONPlaceholder hoặc public/orchids.json	Không phụ thuộc Spring Boot.
HTTP client	Fetch built-in; Axios qua npm	Dùng cả hai để so sánh.

Compatibility note 2026: Vite hiện hành yêu cầu Node 20.19+ hoặc 22.12+. Nếu phòng lab khóa Node 21, dùng project có sẵn hoặc pin Vite 5 thay vì `npm create vite@latest`.

0.4 Pre-class 3-2-1

- 3 điều quan sát được trong một HTTP response.
- 2 lý do UI không nên bị block khi gọi API.
- 1 câu hỏi về Promise/async-await.

0.5 Human pre-check

1. GET khác POST ở mục đích nào?
2. 404 có phải network error không?
3. JSON string khác JavaScript object thế nào?
4. useEffect dùng để làm gì?
5. Nếu request cũ trả về sau request mới thì UI có thể sai thế nào?

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1. Slot 8 → Slot 9

Từ Slot 8	Đã biết	Slot 9 bổ sung
Client–Server	Ai gửi/nhận request	Viết code gửi request bất đồng bộ
HTTP	Method/status/header/body	Xử lý response trong JS
JSON	Representation	Parse/serialize trong client
REST	Endpoint/resource	Fetch/Axios gọi endpoint
DevTools	Quan sát request	Chẩn đoán state/race/cache

2. Định vị Slot 9 trong lộ trình

Slot	Chủ đề	Quan hệ
7	Navigation/Routing nền tảng	Biết URL/client navigation
8	HTTP/JSON/REST	Biết giao tiếp ở mức protocol
9	Fetching & Caching Data	Biết gọi API và quản lý async state
10	React Router & SPA	Ghép URL state + data view
11	Lab 02 + Assignment 01	Tích hợp component/data/navigation
12+	Spring Boot REST	Tự xây backend để React gọi

3. Mục tiêu đầu ra

- Giải thích event loop ở mức mental model cho async I/O.
- Tạo/chain Promise; dùng async/await + try/catch/finally.
- Gọi GET/POST/PUT/DELETE bằng Fetch; kiểm tra `response.ok`.
- Dùng Axios, instance/interceptor cơ bản và xử lý `response.data`.
- Thiết kế Loading/Error/Data state rõ ràng trong React.
- Dùng cleanup/AbortController để tránh stale updates/race conditions.
- Giải thích HTTP cache: freshness, validation, Cache-Control, ETag/If-None-Match, 304.

4. Flow tổng quát

```

HTTP/REST contract (Slot 8)
↓
Async operation starts
↓
Promise: pending
↓
Fetch/Axios sends request
↓
HTTP response arrives
↓
status validation → body parsing
↓
fulfilled / rejected
↓
React loading / error / data
↓
cleanup stale request + cache/revalidation

```

5. Tình huống dẫn nhập – FUNews frontend

Trang News mở ra và cần GET /api/news. Người dùng chuyển category trước khi request cũ hoàn tất. Nếu code chỉ “fetch rồi setState”, response cũ có thể ghi đè response mới. Slot 9 không chỉ dạy cách gọi API mà dạy cách quản lý lifecycle của dữ liệu bất đồng bộ.

6. Tư duy kiến trúc quan trọng

Data fetching is a state machine: Idle → Loading → Success(Data) hoặc Error. Reload/retry tạo một transition mới; cancellation/cleanup loại bỏ kết quả đã stale.

II. TRỌNG TÂM CỐT LỖI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1. JavaScript asynchronous programming – mental model

Trong một JavaScript agent, code JavaScript chạy theo call stack; browser/runtime xử lý timer/network bên ngoài stack và đưa callback/job trở lại event loop khi sẵn sàng. “Async” không có nghĩa mọi JavaScript của bạn chạy song song.

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");

// Quan sát: A, D, C, B
```

Checkpoint: Sinh viên phải giải thích vì sao Promise handler (microtask) chạy trước callback timer trong ví dụ trên, không chỉ nhớ output.

2. Callback và Callback Hell

```
function loadUser(id, callback) {
  setTimeout(() => callback({ id, name: "Lan" })), 500);
}

loadUser(1, user => {
  loadOrders(user.id, orders => {
    loadOrderDetail(orders[0].id, detail => {
      console.log(detail);
    });
  });
});
```

- Callback không “xấu”; vấn đề là nesting, error propagation và control flow khó đọc.
- Promise chuẩn hóa success/error và cho phép composition.

3. Promise: states và settlement

State	Ý nghĩa	Có đối tiếp?
pending	Chưa có kết quả	Có thể settle
fulfilled	Hoàn thành thành công	Không
rejected	Hoàn thành thất bại	Không

```
const delay = ms => new Promise(resolve => setTimeout(resolve, ms));

delay(500)
  .then(() => ({ id: 1, name: "Orchid" })))
  .then(data => console.log(data))
  .catch(err => console.error(err))
  .finally(() => console.log("done"));
```

4. Promise chaining và composition

```
fetch("https://jsonplaceholder.typicode.com/users/1")
  .then(response => {
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return response.json();
  })
  .then(user => fetch(`https://jsonplaceholder.typicode.com/posts?userId=${user.id}`))
  .then(response => {
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return response.json();
  })
  .then(posts => console.log(posts))
  .catch(console.error);
```

API	Dùng khi	Điểm cần nhớ
Promise.all	Tất cả phải thành công	Fail-fast khi một Promise reject
Promise.allSettled	Muốn xem mọi kết quả	Không fail-fast
Promise.race	Kết quả settle đầu tiên	Có thể resolve/reject
Promise.any	Cần fulfilled đầu tiên	Reject nếu tất cả reject

5. async/await + try/catch/finally

```
async function loadUsers() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return await response.json();
  } catch (error) {
    console.error("loadUsers failed", error);
    throw error;
  } finally {
    console.log("request cycle finished");
  }
}
```

Important: `async` function luôn trả Promise. `await` chỉ tạm dừng phần còn lại của chính async function đó; nó không “đóng băng” browser UI.

6. Fetch API – request lifecycle

```
async function getUsers(signal) {
  const response = await fetch(
    "https://jsonplaceholder.typicode.com/users",
    { signal, headers: { Accept: "application/json" } }
  );

  if (!response.ok) {
    throw new Error(`HTTP ${response.status} ${response.statusText}`);
  }
  return response.json();
}
```

Fetch error semantics: `fetch()` không reject chỉ vì server trả 404/500; Promise vẫn fulfilled với `Response`. Vì vậy phải kiểm tra `response.ok` hoặc `response.status`.

7. Fetch CRUD examples

```
// POST
await fetch(url, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(payload)
});

// PUT
await fetch(`${url}/${id}`, { method: "PUT", headers, body: JSON.stringify(payload) });

// DELETE
await fetch(`${url}/${id}`, { method: "DELETE" });
```

8. Response object

Thành phần	Ý nghĩa	Ví dụ
ok	true cho status 200–299	if (!response.ok) ...
status	HTTP status number	404
headers	Response headers	Content-Type, ETag
json()	Đọc body JSON; trả Promise	await response.json()
text()	Đọc body text	await response.text()

9. Axios: client library

```
import axios from "axios";

const api = axios.create({
  baseURL: "https://jsonplaceholder.typicode.com",
  timeout: 5000
});

api.interceptors.response.use(
  response => response,
  error => {
    console.error("API error", error.response?.status ?? error.message);
    return Promise.reject(error);
  }
);

const response = await api.get("/users");
console.log(response.data);
```

10. Fetch vs Axios – phiên bản chuẩn hóa

Tiêu chí	Fetch	Axios
Cài đặt	Built-in browser API	npm package
JSON	Tự gọi response.json()	response.data đã được transform
HTTP 4xx/5xx	Không tự reject	Mặc định reject ngoài 2xx
Cancel	AbortController/AbortSignal	AbortController/AbortSignal
Timeout	Dùng AbortSignal/AbortController pattern	Có `timeout` config
Interceptors	Không có built-in interceptor API	Có request/response interceptors
Upload/download progress	Có thể làm qua streams/XHR patterns tùy môi trường	Có progress callbacks tùy adapter/environment

Do not teach CancelToken: Axios đã hỗ trợ AbortController từ lâu; CancelToken là API deprecated và không nên dùng cho project mới.

11. Loading / Error / Data as explicit state

```
const [data, setData] = useState([]);
const [loading, setLoading] = useState(false);
const [error, setError] = useState(null);
```

State	UI nên làm gì?	Sai lầm phổ biến
loading=true	Spinner/skeleton; tránh double submit	Giữ data cũ mà không nói đang refresh
error!=null	Thông báo + Retry nếu phù hợp	Chỉ console.error
data=[]	Empty state rõ ràng	Nhầm empty với loading
data có item	Render list/detail	Không kiểm tra shape

12. React data fetching với useEffect

```
useEffect(() => {
  const controller = new AbortController();

  async function load() {
    setLoading(true);
    setError(null);
    try {
      const response = await fetch(url, { signal: controller.signal });
      if (!response.ok) throw new Error(`HTTP ${response.status}`);
      setData(await response.json());
    } catch (err) {
      if (err.name !== "AbortError") setError(err.message);
    } finally {
      if (!controller.signal.aborted) setLoading(false);
    }
  }

  load();
  return () => controller.abort();
}, [url]);
```

React Strict Mode: Trong development, React có thể chạy thêm một vòng setup → cleanup → setup để kiểm tra cleanup logic. Đừng dạy `[]` như “chắc chắn chạy đúng một lần” trong mọi tình huống.

13. Race condition

Race xảy ra khi response cũ đến sau response mới và ghi đè state. Cleanup flag hoặc AbortController đều là chiến lược hợp lệ; cancellation còn giúp dừng network/body processing khi môi trường hỗ trợ.

```
// Search term đổi nhanh: url thay đổi → effect cleanup abort request cũ
const url = `/api/search?q=${encodeURIComponent(keyword)}`;
```

14. HTTP caching – browser/protocol layer

Directive/Header	Ý nghĩa đúng	Ví dụ
Cache-Control: max-age=60	Fresh 60 giây	Có thể dùng cached response trong freshness window
no-cache	Có thể lưu, nhưng phải revalidate trước khi reuse	Thường đi cùng validator
no-store	Không lưu response	Dữ liệu nhạy cảm
ETag	Validator đại diện version resource	ETag: "abc123"
If-None-Match	Client gửi validator cũ	Server có thể trả 304
304 Not Modified	Dùng lại cached representation	Response thường không có body mới

Distinguish two caches: HTTP/browser cache khác client state cache trong React. Một custom hook/TanStack Query cache object data trong app; Cache-Control/ETag là semantics ở HTTP layer.

15. Kỹ năng phải đạt

- Trace được Promise/async flow.
- Fetch đúng và kiểm tra status.
- Axios đúng, hiểu response.data/error.response.
- Thiết kế loading/error/data/empty/retry.
- Dùng cleanup/cancel cho request stale.
- Đọc Network tab để xác minh request/cache.
- Giải thích trade-off Fetch vs Axios, browser cache vs app cache.

16. Những hiểu sai phổ biến

Hiểu sai	Điều chỉnh
JavaScript là multi-threaded	JS execution stack là single-threaded; runtime xử lý async I/O/event loop.
async/await biến async thành sync	Chỉ làm control flow dễ đọc; vẫn Promise/async.
fetch 404 sẽ vào catch	Không, cần check response.ok.
no-cache = không lưu	Sai; no-cache yêu cầu revalidate trước reuse.
Axios CancelToken nên dùng	Deprecated; dùng AbortController.
useEffect [] luôn chạy đúng một lần	Strict Mode dev có extra setup/cleanup cycle.
Cache luôn tốt	Có stale/security/consistency trade-offs.

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC TRONG SẢN PHẨM THỰC TẾ

1. Các pattern thực tế

Sản phẩm	Data fetching	Điểm cần xử lý
E-commerce	Products/categories/search	loading, retry, debounce, stale data
News	News list/detail	cache list, fresh detail, pagination
Banking	Account/transactions	no-store/private, error clarity
Social feed	Feed/comments	pagination, race, optimistic UI awareness
Orchid Gallery	Orchid list/detail	service layer + modal + cache

2. Case study mini – FUNews

1. Component mở trang News → set loading.
2. Service gọi GET /api/news.
3. Response 200 → parse JSON → set data.
4. Response 404/500 → error state.
5. Category thay đổi → abort request cũ.
6. Browser may reuse/revalidate response tùy HTTP cache headers.
7. UI render list/empty/error rõ ràng.

3. Service layer boundary

```
// newsService.js
export async function getNews(signal) {
  const response = await fetch("/api/news", { signal });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json();
}
```

Component nên quyết định UI state; service nên tập trung request contract/normalization. Đây là bridge trực tiếp sang Assignment và Spring Boot backend sau này.

IV. NỘI DUNG CHI TIẾT VỀ HƯỚNG DẪN VIỆC THỰC HIỆN VÀ TRIỂN KHAI

A. Demo 1 – Async playground trước React

A1. Mục tiêu

Quan sát thứ tự execution và Promise states trước khi thêm UI.

```
console.log("1 sync");
Promise.resolve().then(() => console.log("3 microtask"));
setTimeout(() => console.log("4 timer"), 0);
console.log("2 sync");
```

A2. Bài kiểm tra

- Dự đoán output trước khi chạy.
- Đổi Promise thành `queueMicrotask`.
- Thêm hai `.then()` và giải thích thứ tự.

B. Demo 2 – Fetch JSONPlaceholder với DevTools

```
async function demoFetch() {  
  const response = await fetch("https://jsonplaceholder.typicode.com/users");  
  console.log(response.status, response.headers.get("content-type"));  
  if (!response.ok) throw new Error(`HTTP ${response.status}`);  
  const users = await response.json();  
  console.table(users);  
}  
  
demoFetch().catch(console.error);
```

1. Mở DevTools → Network.
2. Chạy code/app.
3. Chọn request users.
4. Ghi Method, Status, Request Headers, Response Headers, Response.
5. Thử URL `/users/999999` ; phân biệt empty/not-found behavior của API cụ thể với network failure.

C. Demo 3 – React User List với Loading/Error/Data

```
import { useEffect, useState } from "react";

export default function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();

    async function loadUsers() {
      try {
        setLoading(true);
        setError(null);
        const response = await fetch(
          "https://jsonplaceholder.typicode.com/users",
          { signal: controller.signal }
        );
        if (!response.ok) throw new Error(`HTTP ${response.status}`);
        setUsers(await response.json());
      } catch (err) {
        if (err.name !== "AbortError") setError(err.message);
      } finally {
        if (!controller.signal.aborted) setLoading(false);
      }
    }

    loadUsers();
    return () => controller.abort();
  }, []);

  if (loading) return <p>Loading users...</p>;
  if (error) return <p role="alert">Error: {error}</p>;
  if (users.length === 0) return <p>No users.</p>;

  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

C1. Verify

- Network request xuất hiện.
- Loading thấy được khi throttle Slow 3G.
- Đổi endpoint sai để quan sát Error.
- Không có “set state after stale request” khi component unmount/change URL.

D. Demo 4 – Axios instance + interceptor

```
// apiClient.js
import axios from "axios";

export const apiClient = axios.create({
  baseURL: "https://jsonplaceholder.typicode.com",
  timeout: 5000,
  headers: { Accept: "application/json" }
});

apiClient.interceptors.response.use(
  r => r,
  err => {
    const status = err.response?.status;
    console.error("HTTP client error", status ?? err.message);
    return Promise.reject(err);
  }
);
```

```
// userService.js
import { apiClient } from "../apiClient";

export async function getUsers(signal) {
  const response = await apiClient.get("/users", { signal });
  return response.data;
}
```

E. Demo 5 – client-side cache TTL đơn giản

```
let cache = null;
let cachedAt = 0;
const TTL = 30_000;

export async function getUsersCached(fetcher, { force = false } = {}) {
  const fresh = cache && Date.now() - cachedAt < TTL;
  if (!force && fresh) return cache;

  const data = await fetcher();
  cache = data;
  cachedAt = Date.now();
  return data;
}

export function clearUsersCache() {
  cache = null;
  cachedAt = 0;
}
```

Learning purpose only: Cache module này giúp thấy hit/miss/TTL/invalidation. Production app thường cần library/cache policy có deduplication, retries, invalidation và lifecycle tốt hơn.

F. Demo 6 – quan sát HTTP caching

1. Mở Network tab và request một resource có cache headers.
2. Ghi Cache-Control, ETag hoặc Last-Modified nếu có.
3. Reload và quan sát Size/Status/Timing.
4. Nếu request revalidate bằng If-None-Match và server trả 304, giải thích rằng browser dùng body đã cache.
5. Bật “Disable cache” trong DevTools và so sánh.

G. Break-It Lab

Break	Triệu chứng	Sinh viên phải chẩn đoán
Bỏ response.ok check	404 vẫn đi tiếp parse/render	Fetch semantics
Quên await response.json()	State nhận Promise	Promise vs parsed body
Gọi fetch trực tiếp trong render	Request lặp	Side effect boundary
Effect dependency sai	Loop hoặc stale value	Dependency reasoning
Không cleanup	Response cũ ghi đè	Race condition
Cache không TTL	Data stale vô hạn	Invalidation
Dùng no-cache như no-store	Giải thích cache sai	HTTP semantics

H. Mini Project – User List App

- React component lấy users từ JSONPlaceholder.

- Loading/Error/Data/Empty states.
- Search theo name ở client.
- Reload/Force refresh.
- AbortController khi unmount.
- Chọn Fetch hoặc Axios và viết ADR ngắn giải thích lựa chọn.
- DevTools Evidence: request + status + headers + response.

H1. Acceptance criteria

ID	Tiêu chí	Evidence
AC01	Không request trong render	Code review
AC02	HTTP error được chuyển thành error state	Break endpoint
AC03	Loading nhìn thấy được	Network throttling
AC04	Cleanup/cancel hoạt động	Rapid mount/unmount
AC05	Reload tạo request mới	Network tab
AC06	Cache hit/miss giải thích được	Console/trace
AC07	Sinh viên giải thích flow không nhìn AI	Oral gate

V. KINH NGHIỆM THỰC TIỄN, TIPS

1. Debugging decision tree

```

Không thấy dữ liệu?
├─ Có request trong Network?
│   └─ Không → effect/event/service path?
│       └─ Có → status?
│           ├── 4xx/5xx → contract/auth/path/server
│           └─ 2xx → body/content-type/shape?
├─ Console có exception?
├─ State loading/error/data chuyển đúng không?
├─ Request có bị abort?
└─ Cache có trả data cũ không?
  
```

2. Fetch checklist 6 câu

1. URL/method đúng?
2. Status có được kiểm tra?
3. Body parse đúng kiểu?
4. Error có surfaced lên UI?
5. Request lifecycle có cleanup?
6. Network tab có chứng minh behavior?

3. Axios tips

- Tạo instance thay vì rải baseUrl khắp component.
- Interceptor dùng cho cross-cutting concern, không nhét business logic.
- Không swallow error trong interceptor.
- Dùng `signal` với AbortController; không dùng CancelToken cho code mới.

4. Caching tips

- Cache policy phải trả lời: cache gì, key gì, fresh bao lâu, invalidate lúc nào?
- `no-cache` ≠ “không lưu”.
- Dữ liệu nhạy cảm cần cân nhắc `private` / `no-store` theo threat model.
- DevTools “Disable cache” chỉ phục vụ debug, không phải production policy.

5. useEffect tips

- Effect để synchronize với external system; không dùng effect cho computation thuần có thể tính trong render.

- Cleanup phải đối xứng setup.
- Dependency array mô tả values effect dùng; đừng xóa dependency chỉ để “hết loop”.
- Strict Mode extra cycle là tool phát hiện cleanup bug.

VI. CÁC BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP HỆ THỐNG

A. Bài tập chủ đề

Bài 1 – Async trace

Dự đoán output của sync + Promise + setTimeout; giải thích event loop.

Bài 2 – Promise creation

Promise resolve sau 1s và reject khi input invalid.

Bài 3 – Promise composition

So sánh all/allSettled/race với 3 tác vụ giả lập.

Bài 4 – async/await

Chuyển Promise chain sang async/await + try/catch/finally.

Bài 5 – Fetch GET

GET users, check response.ok, render data.

Bài 6 – Fetch POST

POST JSON, inspect request/response DevTools.

Bài 7 – Axios

Tạo axios instance và gọi cùng endpoint để so sánh.

Bài 8 – HTTP cache trace

Quan sát Cache-Control/ETag/304 trên một resource có hỗ trợ.

Bài 9 – React state machine

Thiết kế loading/error/empty/data UI.

Bài 10 – Race condition

Tạo search request nhanh, tái hiện stale response rồi sửa bằng AbortController.

B. Bài tập tổng hợp – User List Data Client

Xây một React client nhỏ gọi users API, có service layer, loading/error/data, reload, search và cancellation. Không yêu cầu Router hoặc Spring Boot backend.

Deliverable	Yêu cầu
Source	Function components + hooks; Fetch/Axios
README	Cách chạy + lựa chọn Fetch/Axios + cache policy
Evidence	Network screenshots/log: success, error, abort/cache
Test matrix	Ít nhất 10 manual cases
Reflection	3 điều đã hiểu + 1 lỗi khó nhất

C. Quality Gate

1. App chạy.
2. Có service/API boundary.
3. Có loading/error/data/empty.
4. Fetch kiểm tra response.ok hoặc Axios xử lý error đúng.
5. Có cleanup/cancel.
6. DevTools evidence đúng.
7. Sinh viên giải thích cache/race bằng lời.

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ
Asynchronous Programming	Tổ chức công việc không chặn luồng thực thi chờ I/O.	Network request
Call Stack	Nơi JS function đang thực thi.	Function call
Event Loop	Điều phối task/job quay lại JS execution.	Promise/timer
Callback	Function được truyền để gọi sau.	setTimeout callback
Promise	Đại diện kết quả async tương lai.	fetch() returns Promise
pending	Promise chưa settled.	request đang chờ
fulfilled	Promise hoàn thành thành công.	response available
rejected	Promise thất bại.	network error
Promise chaining	Nối <code>.then()</code> theo pipeline.	parse → transform
async	Đánh dấu function trả Promise.	async load()
await	Chờ Promise trong async function.	await fetch()
Fetch API	Web API gửi request.	fetch(url)
Response	Đại diện HTTP response.	status/headers/body
response.ok	true cho 2xx.	HTTP check
Axios	HTTP client library.	axios.get
Interceptor	Hook request/response của Axios.	global error log
AbortController	Controller hủy operation hỗ trợ signal.	cancel fetch
Race condition	Kết quả async đến ngoài thứ tự gây state sai.	search request
useEffect	React Hook synchronize external system.	fetch on dependency

Thuật ngữ	Định nghĩa	Ví dụ
Cleanup	Function đảo/ngừng effect cũ.	controller.abort
Loading state	Request đang chạy.	spinner
Error state	Request/processing lỗi.	alert
Data state	Dữ liệu thành công.	users array
HTTP cache	Reuse response theo HTTP caching rules.	browser cache
Cache-Control	Header điều khiển freshness/storage.	max-age=60
ETag	Validator version resource.	"abc123"
If-None-Match	Conditional request validator.	revalidation
304	Resource không đổi; reuse cached body.	Not Modified
TTL	Thời gian app cache coi data fresh.	30s
Stale data	Cache không còn đủ mới.	expired TTL
JSONPlaceholder	Public fake REST API cho demo.	/users

VIII. RESEARCH BASE VÀ AI BASE LEARNING

A. 10 prompt chuyên sâu kỹ thuật

1. Phân tích Call Stack, task queue và microtask queue trong ví dụ Promise + setTimeout.
2. So sánh Promise.all, allSettled, race và any bằng failure scenarios.
3. Giải thích chính xác vì sao Fetch không reject khi HTTP 404.
4. Phân tích Response body stream và vì sao response.json() là async.
5. Giải thích AbortController/AbortSignal và behavior khi abort trước/sau headers.
6. Phân tích race condition trong React useEffect và hai cách ngăn stale update.
7. Giải thích Strict Mode extra effect cycle và cách viết cleanup đúng.
8. Phân tích ETag/If-None-Match/304 trong HTTP revalidation.
9. So sánh HTTP cache với client-state cache.
10. Phân tích Axios interceptor chain và error propagation.

B. 10 prompt thực tiễn production

1. Thiết kế retry policy: lỗi nào retry, backoff thế nào, lỗi nào không retry?
2. Thiết kế timeout/cancellation cho search-as-you-type.
3. Làm sao chuẩn hóa API errors để UI hiển thị thân thiện?
4. Khi nào nên dùng Axios instance và interceptor ở dự án team?
5. Làm sao tránh network waterfall trong React app?
6. Thiết kế cache key/TTL/invalidation cho news list và news detail.
7. Làm sao phối hợp browser HTTP cache với TanStack Query/SWR?
8. Khi nào cần optimistic update và rollback?
9. Làm sao instrument API latency/error rate phía frontend?
10. Làm sao test race conditions và cancellation một cách lặp lại?

C. 10 prompt cập nhật công nghệ/xu hướng

1. So sánh native Fetch và Axios hiện nay về ergonomics, interceptors, cancellation và compatibility.
2. TanStack Query và SWR giải quyết vấn đề nào so với useEffect thủ công?
3. React frameworks hiện xử lý data fetching khác client effect như thế nào?
4. AbortSignal.timeout và AbortSignal.any có thể đơn giản hóa timeout/cancel ra sao?
5. HTTP cache directives hiện đại: stale-while-revalidate dùng khi nào?
6. Service Worker/Cache API khác HTTP browser cache thế nào?
7. Cursor pagination ảnh hưởng client fetching và cache key thế nào?
8. Streaming/fetch ReadableStream có ứng dụng gì cho progressive UI?
9. GraphQL client cache khác REST query cache thế nào?
10. AI coding assistant nên được kiểm chứng thế nào khi sinh code async?

D. AI Verification Worksheet

Mục	Sinh viên ghi
Prompt	Câu hỏi chính xác đã gửi AI
Claim	AI khẳng định điều gì?
Evidence	MDN/React/Axios docs hoặc DevTools/run output
Experiment	Đã thay đổi code nào để kiểm tra?
Observation	Thấy gì trong Network/Console/UI?
Decision	Giữ/sửa/bỏ đề xuất AI?

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

1. Knowledge check

1. Promise có ba trạng thái nào?
2. async function trả gì?
3. Tại sao Fetch 404 không tự vào catch?
4. response.json() trả gì?
5. Axios khác Fetch ở error semantics mặc định thế nào?
6. no-cache khác no-store thế nào?
7. ETag/If-None-Match dùng để làm gì?
8. Race condition trong fetch là gì?
9. AbortController giải quyết vấn đề gì?
10. Tại sao effect cleanup chạy trước effect mới khi dependency đổi?

2. Skill checklist

Nội dung	Đạt	Chưa đạt	Cần làm lại
Trace async execution	☐	☐	☐
Tạo/chain Promise	☐	☐	☐
Dùng async/await + try/catch/finally	☐	☐	☐
Fetch GET/POST	☐	☐	☐
Check response.ok	☐	☐	☐
Axios GET/POST	☐	☐	☐
Tạo axios instance	☐	☐	☐
Loading/Error/Data/Empty UI	☐	☐	☐
Abort stale request	☐	☐	☐
Quan sát Network	☐	☐	☐
Giải thích Cache-Control/ETag/304	☐	☐	☐
Phân biệt HTTP cache và app cache	☐	☐	☐

3. Mức tự đánh giá

Mức	Biểu hiện
1 – Chưa đạt	Chưa giải thích Promise; gọi API bằng copy-paste; không xử lý error.
2 – Đạt cơ bản	Fetch/Axios được; có loading/error/data; giải thích được flow chính.
3 – Tốt	Xử lý cancellation/race/cache; đọc DevTools; transfer sang API khác và giải thích trade-off.

4. Exit Ticket

- Một điểm khác biệt quan trọng nhất giữa Fetch và Axios?

- Một race condition bạn có thể tái hiện?
- Một bằng chứng trong DevTools chứng minh request/cache behavior?
- Một điều bạn sẽ áp dụng vào Lab 02/Assignment?

PHỤ LỤC A – QUICK REFERENCE

Nhu cầu	Pattern
Fetch + status	<code>`const r=await fetch(url); if(!r.ok) throw ...;`</code>
Parse JSON	<code>`const data=await r.json();`</code>
Axios data	<code>`const {data}=await api.get(path);`</code>
Cancel	<code>`const c=new AbortController(); ... signal:c.signal; c.abort();`</code>
React cleanup	<code>`return () => controller.abort();`</code>
Cache revalidate	<code>`ETag` ↔ `If-None-Match` → 304</code>
Force fresh app data	Invalidate/clear client cache theo policy

PHỤ LỤC B – KẾ HOẠCH GIẢNG 135 PHÚT

Thời lượng	Hoạt động	Kết quả
0–10	Retrieval Slot 8 + async prediction	HTTP/async baseline
10–30	Callback → Promise → event loop	Mental model
30–50	async/await + error handling	Readable async flow
50–75	Fetch + DevTools	Request evidence
75–90	Axios comparison	HTTP client trade-off
90–110	React Loading/Error/Data + Abort	Correct effect lifecycle
110–120	HTTP caching trace	Cache semantics
120–130	Break-It / peer debug	Error diagnosis
130–135	Exit ticket	Learning evidence

PHỤ LỤC C – HUMAN VERIFICATION GATE

1. Vẽ flow từ click Reload đến setData.
2. Tại sao Fetch cần response.ok check?
3. Chứng minh response.json() là Promise.
4. Nếu await ngoài async function thì chuyện gì xảy ra?
5. Promise.all khác allSettled khi một request fail thế nào?
6. Tại sao gọi API trong render gây vấn đề?
7. Effect cleanup chạy lúc nào?
8. Tại sao Strict Mode có thể làm bạn thấy hai request trong development?
9. Race condition trong search xảy ra ra sao?

10. AbortController khác “ignore stale result” ở điểm nào?
11. no-cache có cho browser lưu response không?
12. ETag và 304 tiết kiệm gì?
13. Axios interceptor nên đặt loại logic nào?
14. Khi nào native Fetch là đủ?
15. Khi nào app-level cache cần invalidation?
16. Mở DevTools và chỉ status/header/body của request hiện tại.
17. Đổi endpoint thành lỗi và chứng minh Error UI hoạt động.
18. Throttle network và chứng minh Loading state thật sự tồn tại.
19. Thay URL nhanh và chứng minh request cũ bị abort/ignore.
20. Giải thích một đề xuất AI bạn đã bác bỏ sau khi kiểm chứng.

PHỤ LỤC D – EVIDENCE PACKAGE & GIT CHECKPOINTS

Evidence	Nội dung
E01	Pre-class 3-2-1
E02	Async trace
E03	Promise exercise
E04	Fetch Network evidence
E05	Axios comparison
E06	Loading/Error/Data screenshots
E07	Abort/race evidence
E08	HTTP cache trace
E09	Break-It log
E10	Mini project source
E11	README/ADR Fetch vs Axios
E12	AI Verification Worksheet
E13	Human Gate answers
E14	Exit Ticket

```
git add .
git commit -m "S09 V1 async promise fundamentals"
git commit -m "S09 V2 fetch response handling"
git commit -m "S09 V3 axios service layer"
git commit -m "S09 V4 react loading error data"
git commit -m "S09 V5 abort race cache evidence"
```

TÀI LIỆU THAM KHẢO

- MDN Web Docs – Fetch API / Using the Fetch API.
- MDN Web Docs – Promise và Asynchronous JavaScript.
- MDN Web Docs – HTTP Caching, Cache-Control, ETag, If-None-Match.
- React documentation – useEffect (setup/cleanup, race-condition guidance).

- Axios documentation – Introduction, Interceptors, Cancellation.
- Vite documentation – Getting Started / Node.js compatibility.
- Slide Chapter 08, trang 1–23 và tài liệu Slot 8 của môn SBA301.

Final bridge: Sau Slot 9, sinh viên phải nhìn một màn hình React có dữ liệu và giải thích được không chỉ “API gọi ở đâu”, mà cả request lifecycle, HTTP semantics, UI state, cancellation và cache. Slot 10 mới ghép data view này vào Router/SPA navigation.